

Hoja Referencial MIPS 32

Maivet Pereira

May 2026

1 Aritmetica

add(suma) add \$s1,\$s2,\$s3 # \$s1 = \$s2 + \$s3
subtract(resta) sub \$s1,\$s2,\$s3 # \$s1 = \$s2 - \$s3
add immediate addi \$s1,\$s2,100 # \$s1 = \$s2 + 100

2 Transferencia de Datos

w = Word (Palabra entera = 4 bytes = El tamaño de un int normal).

h = Halfword (Media palabra = 2 bytes = El tamaño de un short).

b = Byte (Un solo byte = 1 byte = El tamaño de un char).

load word lw \$s1, 4(\$s2)\$s1 = TRAE un entero desde la RAM (dirección \$s2 + 100)
store word sw \$s1, 8(\$s2) # GUARDA el entero de \$s1 en la RAM (dirección \$s2 + 100)
load halfword lh \$s1, 100(\$s2) # \$s1 = TRAE medio entero desde la RAM (dirección \$s2 + 100)
store halfword sh \$s1, 100(\$s2) # GUARDA solo la mitad de \$s1 en la RAM (dirección \$s2 + 100)
load byte lb \$s1, 100(\$s2) # \$s1 = TRAE un solo byte desde la RAM (dirección \$s2 + 100)
store byte sb \$s1, 100(\$s2) # GUARDA solo el último byte de \$s1 en la RAM (dirección \$s2 + 100)
load upper immediate lui \$s1, 100 # Mete el número 100 en la parte ALTA (izquierda) de \$s1 (y llena de ceros el resto)

3 Logica

and and \$s1, \$s2, \$s3 # \$s1 = \$s2 Y \$s3 (bit a bit, da 1 solo si ambos bits son 1)
or or \$s1, \$s2, \$s3 # \$s1 = \$s2 O \$s3 (bit a bit, da 1 si al menos uno de los bits es 1)
xor xor \$s1, \$s2, \$s3 # \$s1 = \$s2 o exclusivo \$s3 (da 1 si los bits son diferentes)
nor nor \$s1, \$s2, \$s3 # \$s1 = NO (\$s2 O \$s3) (invierte el resultado del O, util para hacer un NOT)
and immediate andi \$s1, \$s2, 100 # \$s1 = \$s2 Y el número 100 (para comparar un registro con una constante)
or immediate ori \$s1, \$s2, 100 # \$s1 = \$s2 O el número 100 (se usa mucho junto a lui para cargar constantes grandes)
shift left logical sll \$s1, \$s2, 2 # \$s1 = Desplaza los bits de \$s2 a la IZQUIERDA 2 veces (Equivale a multiplicar por 4)
shift right logical srl \$s1, \$s2, 2 # \$s1 = desplaza los bits de \$s2 a la DERECHA 2 veces (equivale a dividir entre 4)

4 Salto Condicional

branch if equal beq \$s1, \$s2, label # si (\$s1 == \$s2) salta a la etiqueta label
branch if not equal bne \$s1, \$s2, label # si (\$s1 != \$s2) salta a la etiqueta label

5 Salto Incondicional

jump j label # salto incondicional (va directo a la etiqueta label sin preguntar)

jump and link jal funcion # llama a una funcion y guarda la direccion de retorno en \$ra

jump register jr \$ra # salta a la direccion guardada en \$ra (se usa para volver de una funcion)

set on less than slt \$s1, \$s2, \$s3 # si (\$s2 < \$s3) \$s1 = 1; si no \$s1 = 0 (sirve para los if de menor que)

set on less than immediate slti \$s1, \$s2, 100 # si (\$s2 < 100) \$s1 = 1; si no \$s1 = 0 (compara un registro con un numero)

6 Formato Basico de Instruccion

R

cod oper (6 bits)	rs (5 bits)	rt (5 bits)	rd (5 bits)	desplaz (5 bits)	func (6 bits)
-------------------	-------------	-------------	-------------	------------------	---------------

I

cod oper (6 bits)	rs (5 bits)	rt (5 bits)	inmediato (16 bits)
-------------------	-------------	-------------	---------------------

J

cod oper (6 bits)	direccion (26 bits)
-------------------	---------------------